

230229P_A_01

September 14, 2026

0.1 Hansindu M.M.A.D. – 230229P

0.1.1 Question 1

```
[160]: import cv2
import matplotlib.pyplot as plt
import numpy as np

def intensity_transform(im, breakpoints):
    # Applies a piecewise linear intensity transformation using a lookup table
    ↪ (LUT)

    points = np.array(breakpoints)
    x_in, y_out = points[:, 0], points[:, 1]

    lut = np.interp(np.arange(256), x_in, y_out).astype(np.uint8)

    transformed_im = cv2.LUT(im, lut)
    return transformed_im, lut

breakpoints_1= [[0, 0], [50, 50], [100, 150], [150, 255], [255, 255]]

im = cv2.imread("images/q1.jpeg", cv2.IMREAD_GRAYSCALE)

transformed_im_1, lut_1 = intensity_transform(im, breakpoints_1)

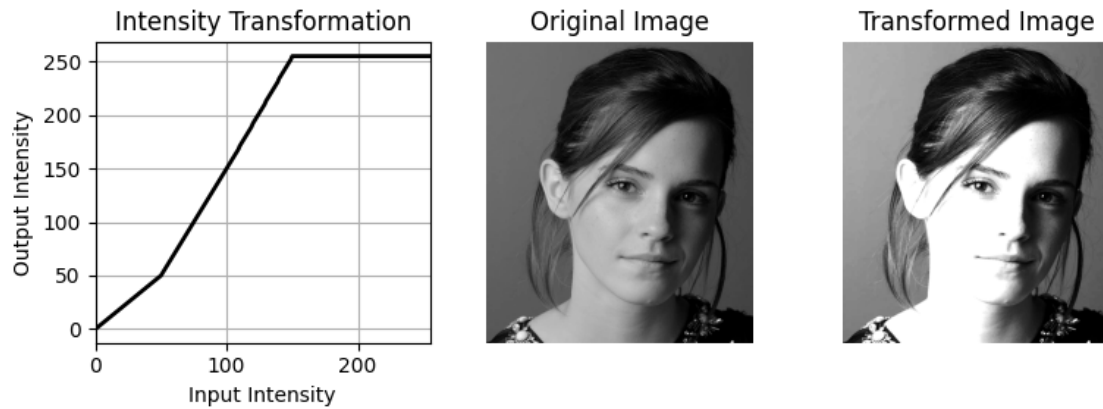
# Plotting transformation curve and side-by-side images
fig, ax = plt.subplots(1, 3, figsize=(8, 3))

ax[0].plot(np.arange(256), lut_1, linestyle='solid', linewidth= 2, color=
    ↪ 'black'); ax[0].set_title('Intensity Transformation')
ax[0].set_xlabel('Input Intensity'); ax[0].set_ylabel('Output Intensity');
    ↪ ax[0].set_xlim([0, 255]); ax[0].grid(True)

ax[1].imshow(im, cmap="gray"); ax[1].set_title('Original Image'); ax[1].
    ↪ axis('off')
```

```
ax[2].imshow(transformed_im_1, cmap="gray"); ax[2].set_title('Transformed_
↳Image'); ax[2].axis('off')

plt.tight_layout()
plt.show()
```



```
[161]: breakpoints_2= [[0, 0], [50, 50], [50, 100], [150, 255], [150, 150], [255, 255]]

transformed_im_2, lut_2 = intensity_transform(im, breakpoints_2)

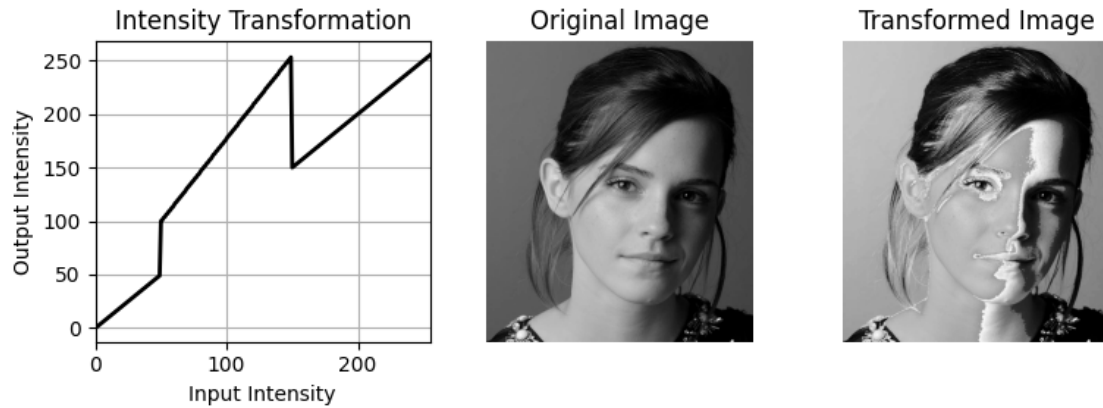
# Plotting transformation curve and side-by-side images
fig, ax = plt.subplots(1, 3, figsize=(8, 3))

ax[0].plot(np.arange(256), lut_2, linestyle='solid', linewidth= 2, color='
↳black'); ax[0].set_title('Intensity Transformation')
ax[0].set_xlabel('Input Intensity'); ax[0].set_ylabel('Output Intensity');
↳ax[0].set_xlim([0, 255]); ax[0].grid(True)

ax[1].imshow(im, cmap="gray"); ax[1].set_title('Original Image'); ax[1].
↳axis('off')

ax[2].imshow(transformed_im_2, cmap="gray"); ax[2].set_title('Transformed_
↳Image'); ax[2].axis('off')

plt.tight_layout()
plt.show()
```



```
[162]: breakpoints_3= [[0, 0], [50, 50], [200, 150], [255, 255]]

transformed_im_3, lut_3 = intensity_transform(im, breakpoints_3)

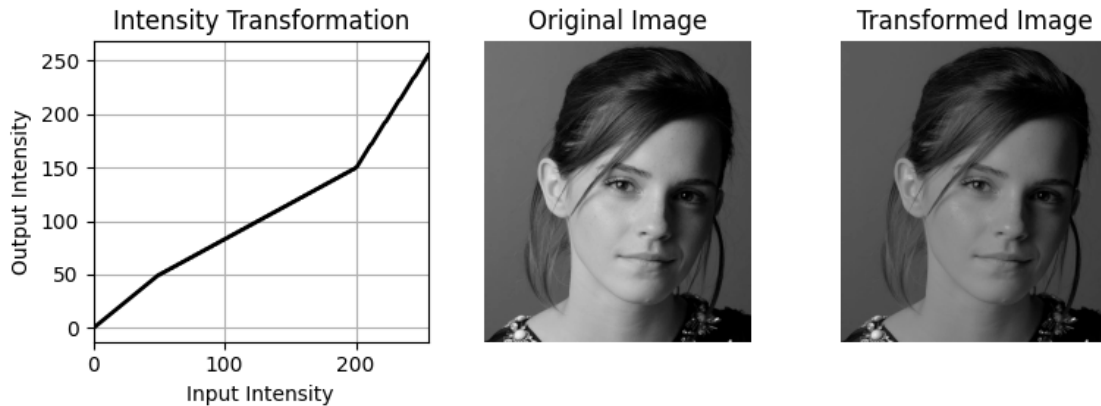
# Plotting transformation curve and side-by-side images
fig, ax = plt.subplots(1, 3, figsize=(8, 3))

ax[0].plot(np.arange(256), lut_3, linestyle='solid', linewidth= 2, color='black'); ax[0].set_title('Intensity Transformation')
ax[0].set_xlabel('Input Intensity'); ax[0].set_ylabel('Output Intensity'); ax[0].set_xlim([0, 255]); ax[0].grid(True)

ax[1].imshow(im, cmap="gray"); ax[1].set_title('Original Image'); ax[1].axis('off')

ax[2].imshow(transformed_im_3, cmap="gray"); ax[2].set_title('Transformed Image'); ax[2].axis('off')

plt.tight_layout()
plt.show()
```



0.1.2 Question 2

```
[164]: breakpoints_4= [[0, 0], [184, 0], [185, 255],[255, 255]]
breakpoints_5= [[0, 0], [139, 0], [140, 255],[184, 255], [185, 0], [255, 0]]

im_1 = cv2.imread("images/q2.jpeg", cv2.IMREAD_GRAYSCALE)

transformed_im_4, lut_4 = intensity_transform(im_1, breakpoints_4)
transformed_im_5, lut_5 = intensity_transform(im_1, breakpoints_5)

# plotting histogram of the original image to identify the intensity
↳distribution
plt.figure(figsize=(5,3)), plt.hist(im_1.ravel(), bins=256, range=[0,256])
plt.xlabel("Intensity"), plt.ylabel("Frequency")
plt.xticks(np.arange(0, 256, 10), rotation=90), plt.xlim([0, 255]), plt.
↳grid(True), plt.show()

# Plotting transformation curve and side-by-side images
fig, ax = plt.subplots(2, 3, figsize=(8, 4))

ax[0][0].plot(np.arange(256), lut_4, linestyle='solid', linewidth= 2, color=
↳'black'), ax[0][0].set_title('Intensity Transformation for White Matter
↳Accentuation', fontsize=8)
ax[0][0].set_xlabel('Input Intensity'), ax[0][0].set_ylabel('Output
↳Intensity'), ax[0][0].set_xlim([0, 255]), ax[0][0].grid(True)

ax[0][1].imshow(im_1, cmap="gray"), ax[0][1].set_title('Original Image',
↳fontsize=10), ax[0][1].axis('off')

ax[0][2].imshow(transformed_im_4, cmap="gray"), ax[0][2].set_title('Transformed
↳Image', fontsize=10), ax[0][2].axis('off')
```

```

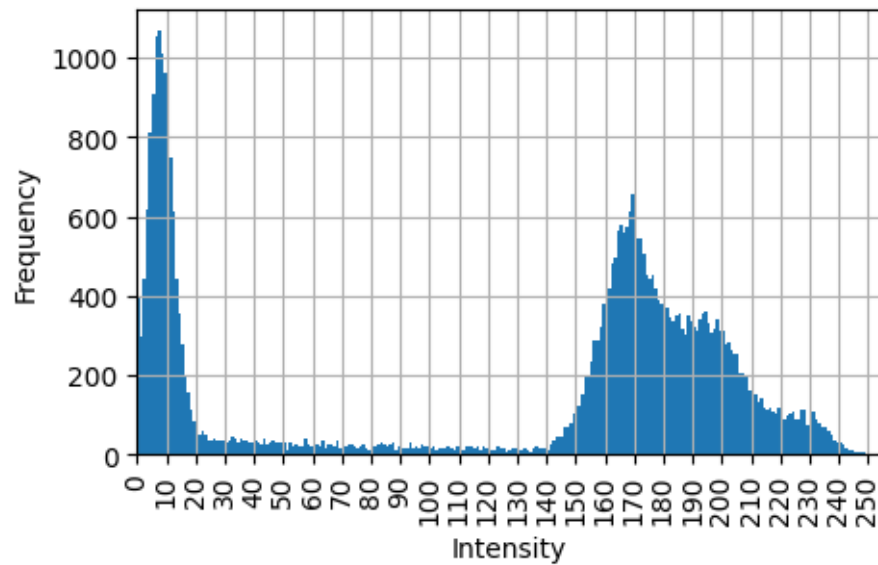
ax[1][0].plot(np.arange(256), lut_5, linestyle='solid', linewidth= 2, color=
    ↳'black'), ax[1][0].set_title('Intensity Transformation for Gray Matter_
    ↳Accentuation', fontsize=8)
ax[1][0].set_xlabel('Input Intensity'), ax[1][0].set_ylabel('Output_
    ↳Intensity'), ax[1][0].set_xlim([0, 255]), ax[1][0].grid(True)

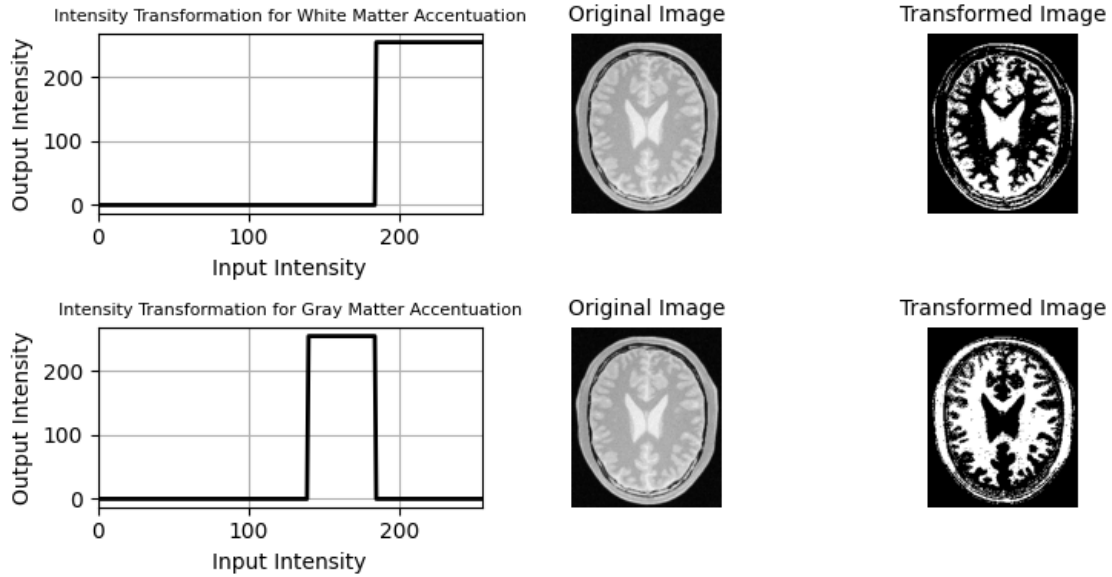
ax[1][1].imshow(im_1, cmap="gray"), ax[1][1].set_title('Original Image',
    ↳fontsize=10), ax[1][1].axis('off')

ax[1][2].imshow(transformed_im_5, cmap="gray"), ax[1][2].set_title('Transformed_
    ↳Image', fontsize=10), ax[1][2].axis('off')

plt.tight_layout()
plt.show()

```





select the range from 140 to 185 as gray matter range and the range from 185 to 255 as white matter range.

0.1.3 Question 3

```
[165]: im_bgr = cv2.imread("images/WhatsApp Image 2026-09-09 at 20.02.44 (1).jpeg")

# separating the L*a*b* channels
L, a, b = cv2.split(cv2.cvtColor(im_bgr, cv2.COLOR_BGR2LAB))

# Applying Gamma correction to the L channel
gamma = 0.7
print(f"Applied Gamma value: {gamma}")

lut_gamma = np.array([((i / 255.0) ** gamma) * 255 for i in range(256)]).
    ↳astype(np.uint8)
L_corrected = cv2.LUT(L, lut_gamma)

# merge corrected L channel with original a and b channels
im_lab_corrected = cv2.merge([L_corrected, a, b])

fig, ax = plt.subplots(2, 2, figsize= (8,4))

# Plotting the original image
ax[0][0].imshow(cv2.cvtColor(im_bgr, cv2.COLOR_BGR2RGB)), ax[0][0].
    ↳set_title('Original Image', fontsize=10), ax[0][0].axis('off')
```

```

# Plotting Gamma corrected image
ax[0][1].imshow(cv2.cvtColor(im_lab_corrected, cv2.COLOR_LAB2RGB)), ax[0][1].
    ↳set_title('Gamma Corrected Image', fontsize=10), ax[0][1].axis('off')

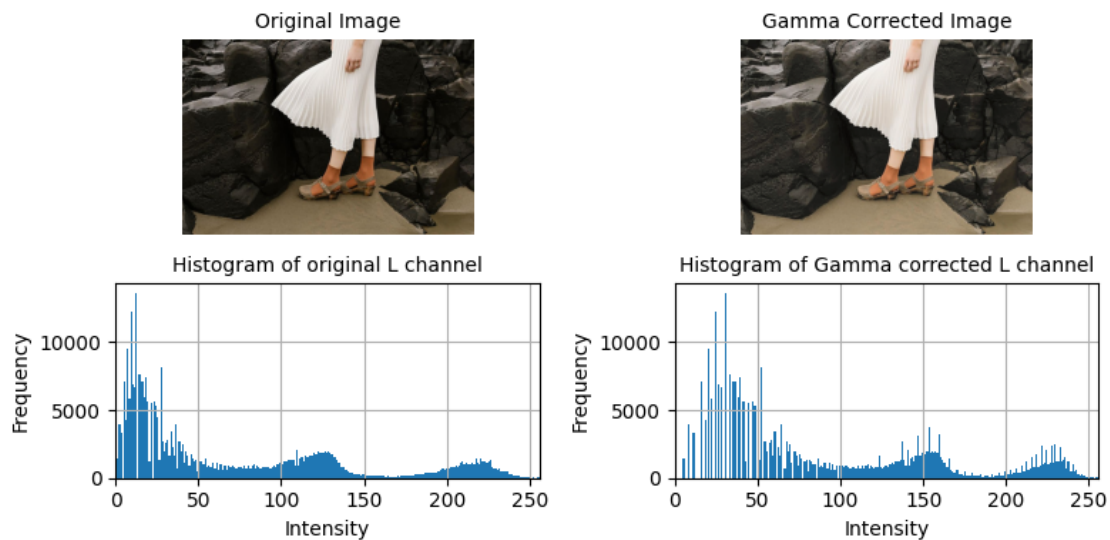
# plotting the histogram of original L channel
ax[1][0].hist(L.ravel(), bins= 256, range= [0, 256]), ax[1][0].
    ↳set_title('Histogram of original L channel', fontsize=10), ax[1][0].
    ↳set_xlabel('Intensity')
ax[1][0].set_ylabel('Frequency'), ax[1][0].set_xlim([0, 256]), ax[1][0].
    ↳grid(True)

# plotting the histogram of Gamma corrected L channel
ax[1][1].hist(L_corrected.ravel(), bins= 256, range= [0, 256]), ax[1][1].
    ↳set_title('Histogram of Gamma corrected L channel', fontsize=10), ax[1][1].
    ↳set_xlabel('Intensity')
ax[1][1].set_ylabel('Frequency'), ax[1][1].set_xlim([0, 256]), ax[1][1].
    ↳grid(True)

plt.tight_layout()
plt.show()

```

Applied Gamma value: 0.7



0.1.4 Question 4

```

[166]: im_2 = cv2.imread("images/WhatsApp Image 2026-09-09 at 20.02.441.jpeg")

# converting the image to HSV color space

```

```
im_hsv = cv2.cvtColor(im_2, cv2.COLOR_BGR2HSV)
```

```
# separate into planes
```

```
hue_plane = im_hsv[:, :, 0]
```

```
saturation_plane = im_hsv[:, :, 1]
```

```
value_plane = im_hsv[:, :, 2]
```

```
[167]: # Apply the aforementioned intensity transformation to the saturation plane
```

```
a_val = 0.95
```

```
sigma = 70.0
```

```
x = np.arange(256, dtype=np.float32)
```

```
boost = a_val * 128.0 * np.exp(-((x - 128.0) ** 2) / (2.0 * (sigma**2)))
```

```
fx = np.minimum(x + boost, 255.0).astype(np.uint8)
```

```
corrected_saturation_plane = cv2.LUT(saturation_plane, fx)
```

```
print(f"Applied a value: {a_val}")
```

Applied a value: 0.95

```
[168]: # recombining three planes back to get vibrance-enhanced image
```

```
im_hsv_corrected = cv2.merge([hue_plane, corrected_saturation_plane,   
    ↪value_plane])
```

```
# showing all plots
```

```
fig, ax = plt.subplots(1,2, figsize= (8, 3))
```

```
ax[0].imshow(cv2.cvtColor(im_2, cv2.COLOR_BGR2RGB)), ax[0].set_title('Original_  
    ↪Image', fontsize=10), ax[0].axis('off')
```

```
ax[1].imshow(cv2.cvtColor(im_hsv_corrected, cv2.COLOR_HSV2RGB)), ax[1].  
    ↪set_title('Vibrance-enhanced Image', fontsize=10), ax[1].axis('off'), plt.  
    ↪show()
```

```
plt.figure(figsize=(5,3)); plt.plot(np.arange(256), fx, linestyle='solid',   
    ↪linewidth= 2, color= 'black')
```

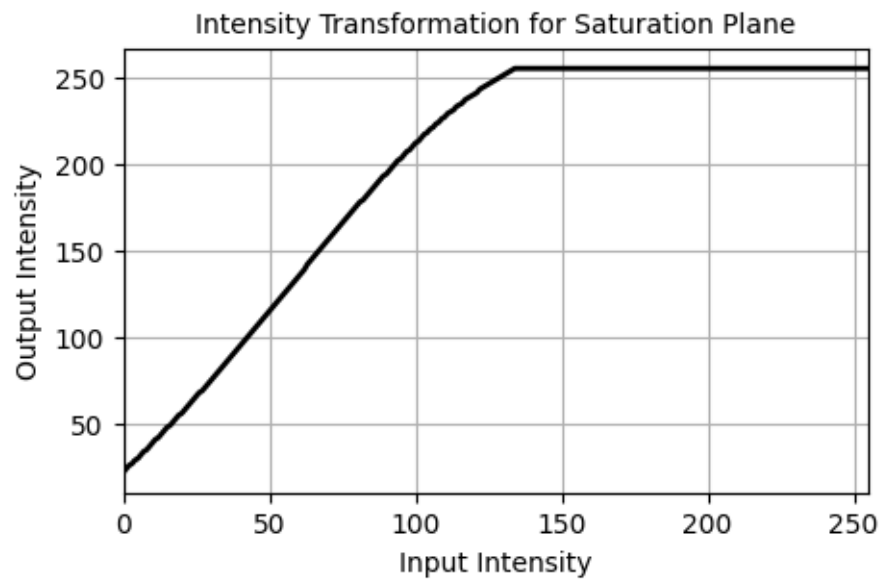
```
plt.title('Intensity Transformation for Saturation Plane', fontsize=10); plt.  
    ↪xlabel('Input Intensity')
```

```
plt.ylabel('Output Intensity'); plt.xlim([0, 255]); plt.grid(True); plt.show()
```

Original Image



Vibrance-enhanced Image



0.1.5 Question 5

```
[144]: def custom_histogram_equalization(im_gray):  
    # Equalize the histogram of grayscale image  
  
    # Compute histogram  
    hist, bin_edges = np.histogram(im_gray.flatten(), bins=256, range=(0, 256))  
  
    # Compute PMF and CDF  
    pmf = hist / im_gray.size  
    cdf = np.cumsum(pmf)  
  
    # Map CDF to intensity range  
    lookup_table = np.round(cdf * 255).astype(np.uint8)
```

```

    # Apply mapping
    equalized_im = lookup_table[im_gray]
    return equalized_im

im_3 = cv2.imread("images/shells.tif", cv2.IMREAD_GRAYSCALE)

equalaized_im_3 = custom_histogram_equalization(im_3)

fig, ax = plt.subplots(2, 2, figsize= (8,5))

# plotting the original image
ax[0][0].imshow(im_3, cmap='gray'), ax[0][0].set_title('Original Image',
    ↪fontsize=10), ax[0][0].axis('off')

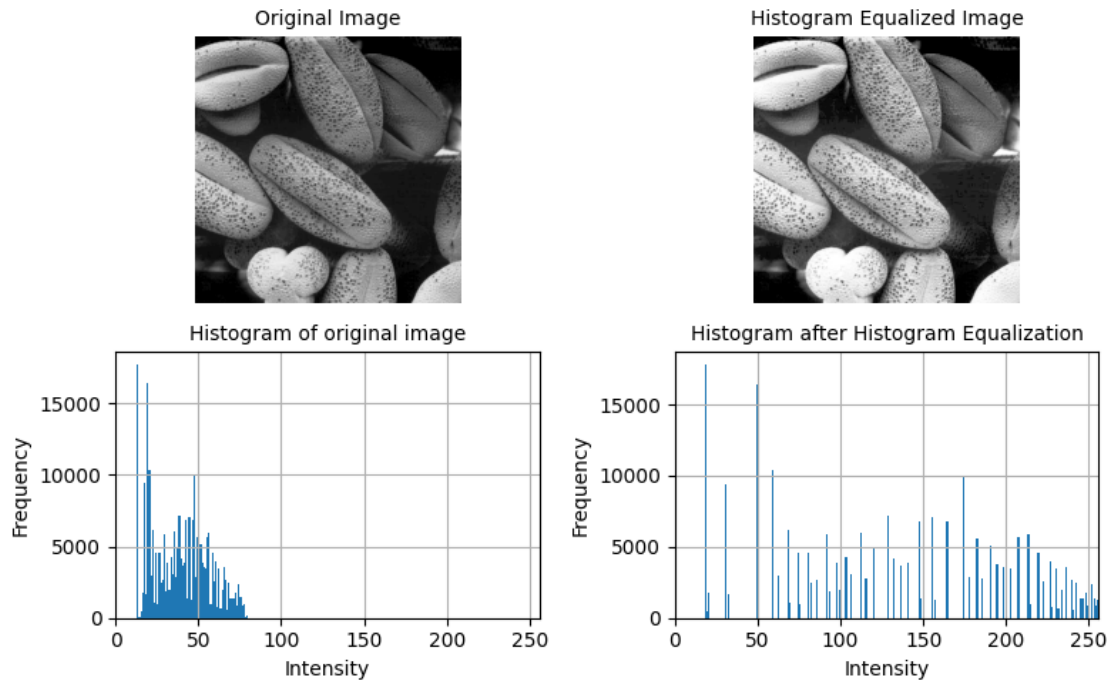
# Plotting histogram qualized image
ax[0][1].imshow(equalaized_im_3, cmap='gray'), ax[0][1].set_title('Histogram_
    ↪Equalized Image', fontsize=10), ax[0][1].axis('off')

# plotting the Histogram of original image
ax[1][0].hist(im_3.flatten(), bins= 256, range= [0, 256]), ax[1][0].
    ↪set_title('Histogram of original image', fontsize=10)
ax[1][0].set_xlabel('Intensity'), ax[1][0].set_ylabel('Frequency'), ax[1][0].
    ↪set_xlim([0, 256]), ax[1][0].grid(True)

# plotting the Histogram after Histogram Equalization
ax[1][1].hist(equalaized_im_3.flatten(), bins= 256, range= [0, 256]), ax[1][1].
    ↪set_title('Histogram after Histogram Equalization', fontsize=10)
ax[1][1].set_xlabel('Intensity'), ax[1][1].set_ylabel('Frequency'), ax[1][1].
    ↪set_xlim([0, 256]), ax[1][1].grid(True)

plt.tight_layout()
plt.show()

```



0.1.6 Question 6

```
[170]: im_4 = cv2.imread("images/WhatsApp Image 2026-09-09 at 20.02.44 (2).jpeg")

# converting the image to HSV color space
im_4_hsv = cv2.cvtColor(im_4, cv2.COLOR_BGR2HSV)

# separate into planes
hue_plane = im_4_hsv[:, :, 0]
saturation_plane = im_4_hsv[:, :, 1]
value_plane = im_4_hsv[:, :, 2]

# showing all planes of HSV color space
fig, ax = plt.subplots(2,2, figsize= (6, 4))

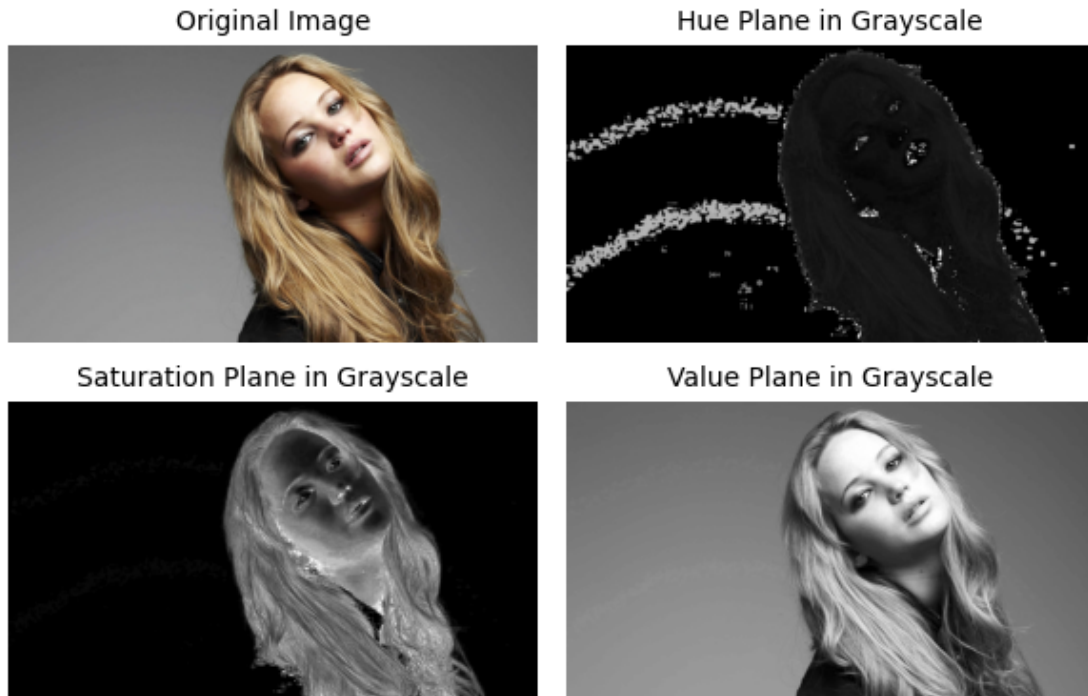
ax[0][0].imshow(cv2.cvtColor(im_4, cv2.COLOR_BGR2RGB)), ax[0][0].
    ↪set_title('Original Image', fontsize=10), ax[0][0].axis('off')

ax[0][1].imshow(hue_plane, cmap='gray'), ax[0][1].set_title('Hue Plane in_
    ↪Grayscale', fontsize=10), ax[0][1].axis('off')

ax[1][0].imshow(saturation_plane, cmap='gray'), ax[1][0].set_title('Saturation_
    ↪Plane in Grayscale', fontsize=10), ax[1][0].axis('off')
```

```
ax[1][1].imshow(value_plane, cmap='gray'), ax[1][1].set_title('Value Plane in_
↳Grayscale', fontsize=10), ax[1][1].axis('off')

plt.tight_layout()
plt.show()
```

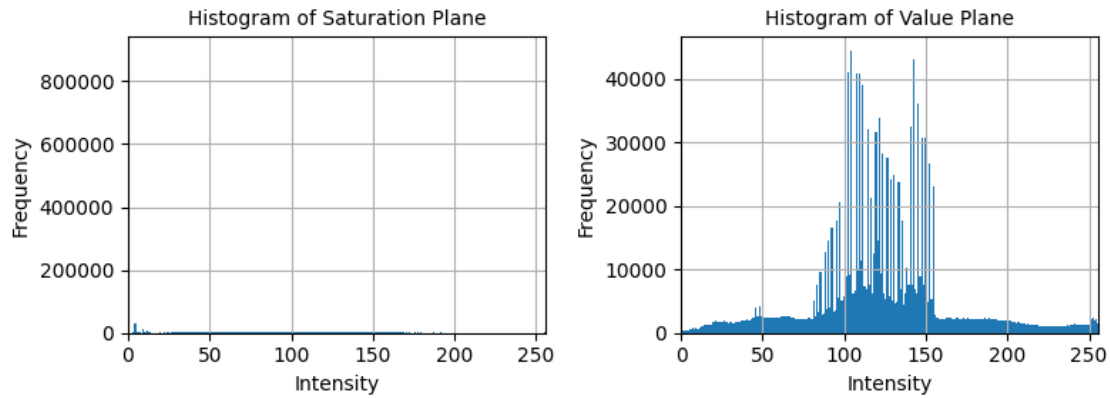


```
[171]: fig, ax = plt.subplots(1, 2, figsize= (8,3))

# plotting the Histogram of Saturation Plane
ax[0].hist(saturation_plane.flatten(), bins= 256, range= [0, 256]), ax[0].
↳set_title('Histogram of Saturation Plane', fontsize=10)
ax[0].set_xlabel('Intensity'), ax[0].set_ylabel('Frequency'), ax[0].
↳set_xlim([0, 256]), ax[0].grid(True)

# plotting the Histogram of Value Plane
ax[1].hist(value_plane.flatten(), bins= 256, range= [0, 256]), ax[1].
↳set_title('Histogram of Value Plane', fontsize=10)
ax[1].set_xlabel('Intensity'), ax[1].set_ylabel('Frequency'), ax[1].
↳set_xlim([0, 256]), ax[1].grid(True)

plt.tight_layout()
plt.show()
```



Selected the Saturation plane to extract the foreground mask after looking at the grayscale images and histograms of 3 planes.

```
[172]: # threshold saturation plane
ret_value, mask = cv2.threshold(saturation_plane, 14, 255, cv2.THRESH_BINARY)
binary_mask = (mask > 0).astype(np.uint8)

# display the foreground mask
plt.figure(figsize=(4, 3)); plt.imshow(binary_mask, cmap='gray')
plt.title('Foreground Mask'); plt.axis('off'); plt.show()
```



```
[173]: # Extract foreground on the Value plane
foreground = cv2.bitwise_and(value_plane, value_plane, mask=binary_mask)

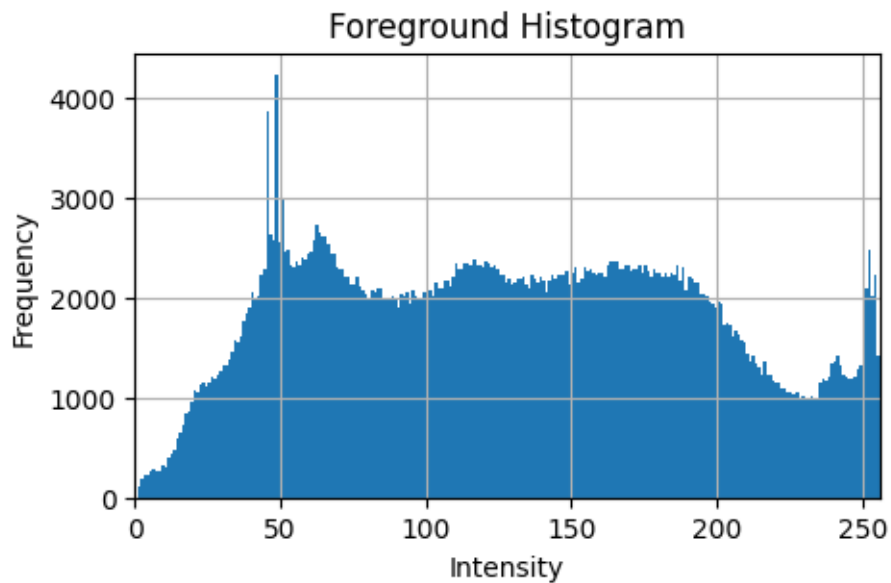
plt.figure(figsize=(4, 3)); plt.imshow(foreground, cmap='gray')
plt.title('Foreground (Value Plane)'); plt.axis('off'); plt.show()
```

Foreground (Value Plane)



```
[156]: # Compute histogram of the foreground pixels
fg_pixels = value_plane[binary_mask == 1]
hist, bin_edges = np.histogram(fg_pixels, bins=256, range=(0, 256))

# Histogram of the foreground pixels
plt.figure(figsize=(5, 3)); plt.hist(fg_pixels, bins=256, range=[0, 256]); plt.
    title('Foreground Histogram')
plt.xlabel('Intensity'); plt.ylabel('Frequency'); plt.xlim([0, 256]); plt.
    grid(True); plt.show()
```



```
[175]: # calculate the CDF of the foreground histogram
cdf_fg = np.cumsum(hist / fg_pixels.size)
```

```

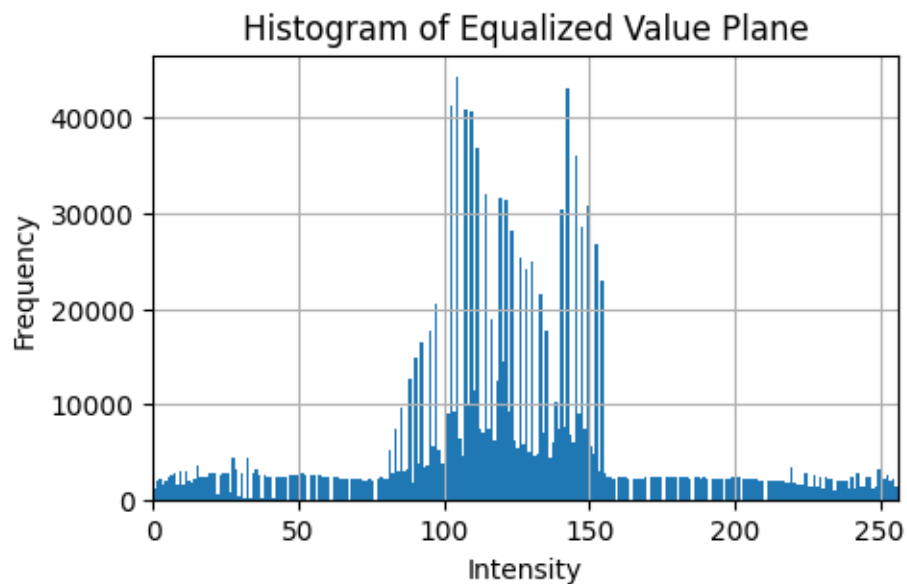
lut_fg = np.round(cdf_fg * 255).astype(np.uint8)

# Apply equalization to foreground Value channel
value_equalized = value_plane.copy()
value_equalized[binary_mask == 1] = lut_fg[value_plane[binary_mask == 1]]

plt.figure(figsize=(5, 3)); plt.hist(value_equalized.flatten(), bins= 256,
    ↪range= [0, 256]); plt.title('Histogram of Equalized Value Plane')
plt.xlabel('Intensity'); plt.ylabel('Frequency'); plt.xlim([0, 256]); plt.
    ↪grid(True); plt.show()

im_4_fg_equalized = cv2.merge([hue_plane, saturation_plane, value_equalized])

```



```

[176]: fig, ax = plt.subplots(1, 2, figsize= (6,4))

# plotting the original image
ax[0].imshow(cv2.cvtColor(im_4, cv2.COLOR_BGR2RGB)); ax[0].set_title('Original_
    ↪Image'); ax[0].axis('off')

# Plotting histogram qualized image
ax[1].imshow(cv2.cvtColor(im_4_fg_equalized, cv2.COLOR_HSV2RGB)); ax[1].
    ↪set_title('Histogram Equalized Image'); ax[1].axis('off')

plt.tight_layout(); plt.show()

```

Original Image



Histogram Equalized Image



0.1.7 Question 7

```
[183]: im_5 = cv2.imread("images/WhatsApp Image 2026-09-09 at 20.02.43 (2).jpeg", cv2.
        ↪IMREAD_GRAYSCALE)

# Standard Sobel Kernels (3x3)
K_x = np.array([[ -1,  0,  1], [-2,  0,  2], [-1,  0,  1]], dtype=np.float32)
K_y = np.array([[ -1, -2, -1], [ 0,  0,  0], [ 1,  2,  1]], dtype=np.float32)

# OpenCV filter2D
Gx_a = cv2.filter2D(im_5, cv2.CV_64F, K_x)
Gy_a = cv2.filter2D(im_5, cv2.CV_64F, K_y)
G_a = np.hypot(Gx_a, Gy_a)

fig, axes = plt.subplots(1, 4, figsize=(8, 2))

# Original Grayscale Image
axes[0].imshow(im_5, cmap='gray'); axes[0].set_title('Original Grayscale_
↪Image', fontsize=10); axes[0].axis('off')

# Horizontal Gradient
axes[1].imshow(np.abs(Gx_a), cmap='gray'); axes[1].set_title('Gx_a (Horizontal_
↪Gradients)', fontsize=10); axes[1].axis('off')

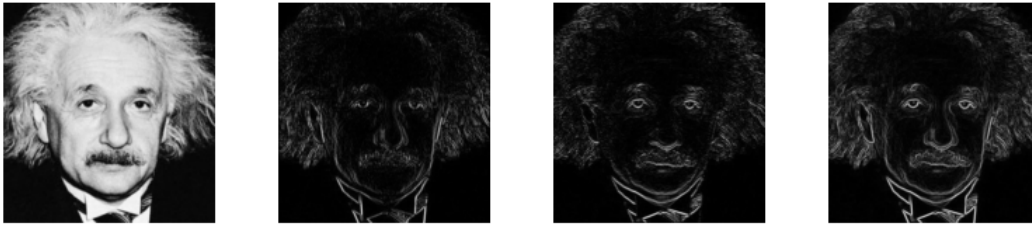
# Vertical Gradient
axes[2].imshow(np.abs(Gy_a), cmap='gray'); axes[2].set_title('Gy_a (Vertical_
↪Gradients)', fontsize=10); axes[2].axis('off')

# Total Gradient Magnitude
axes[3].imshow(G_a, cmap='gray'); axes[3].set_title('G_a (Overall Edge_
↪Magnitude)', fontsize=10); axes[3].axis('off')

plt.tight_layout()
```

```
plt.show()
```

Original Grayscale Image Gx_a (Horizontal Gradients) Gy_a (Vertical Gradients) G_a (Overall Edge Magnitude)



```
[153]: def custom_conv2d(image, kernel):
        # 2D Convolution Function

        kh, kw = kernel.shape
        pad_h, pad_w = kh // 2, kw // 2

        # Convert image to float64 for high-precision arithmetic
        padded = np.pad(image.astype(np.float64), ((pad_h, pad_h), (pad_w, pad_w)),
        ↪mode='edge')
        output = np.zeros_like(image, dtype=np.float64)

        # Correct loop bounds over image height and width
        for i in range(image.shape[0]):
            for j in range(image.shape[1]):
                region = padded[i : i + kh, j : j + kw]
                output[i, j] = np.sum(region * np.flip(kernel))
        return output

Gx_b = custom_conv2d(im_5, K_x)
Gy_b = custom_conv2d(im_5, K_y)
G_b = np.hypot(Gx_b, Gy_b)
```

(c) Separable 1D Filtering (Sobel Kernel Factorization)

$$\text{Sobel X} = [1, 2, 1]^T * [-1, 0, 1]$$

$$\text{Sobel Y} = [-1, 0, 1]^T * [1, 2, 1]$$

```
[184]: k_smoothing = np.array([[1], [2], [1]], dtype=np.float32)
        k_diff = np.array([[1, 0, -1]], dtype=np.float32)

        # Apply 1D horizontal pass followed by 1D vertical pass
        Gx_c = cv2.sepFilter2D(im_5, cv2.CV_64F, k_diff, k_smoothing)
        Gy_c = cv2.sepFilter2D(im_5, cv2.CV_64F, k_smoothing.T, k_diff.T)
```

```

G_c = np.hypot(Gx_c, Gy_c)

fig, axes = plt.subplots(2, 3, figsize=(7, 4))

# Horizontal Gradient
axes[0][0].imshow(np.abs(Gx_b), cmap='gray'); axes[0][0].set_title('Gx_b_
↳(Horizontal Gradients)', fontsize=10); axes[0][0].axis('off')

# Vertical Gradient
axes[0][1].imshow(np.abs(Gy_b), cmap='gray'); axes[0][1].set_title('Gy_b_
↳(Vertical Gradients)', fontsize=10); axes[0][1].axis('off')

# Total Gradient Magnitude
axes[0][2].imshow(G_b, cmap='gray'); axes[0][2].set_title('G_b (Overall Edge_
↳Magnitude)', fontsize=10); axes[0][2].axis('off')

axes[1][0].imshow(np.abs(Gx_c), cmap='gray'); axes[1][0].set_title('Gx_c_
↳(Horizontal Gradients)', fontsize=10); axes[1][0].axis('off')

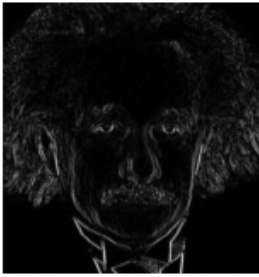
# Vertical Gradient
axes[1][1].imshow(np.abs(Gy_c), cmap='gray'); axes[1][1].set_title('Gy_c_
↳(Vertical Gradients)', fontsize=10); axes[1][1].axis('off')

# Total Gradient Magnitude
axes[1][2].imshow(G_c, cmap='gray'); axes[1][2].set_title('G_c (Overall Edge_
↳Magnitude)', fontsize=10); axes[1][2].axis('off')

plt.tight_layout(); plt.show()

```

Gx_b (Horizontal Gradients)



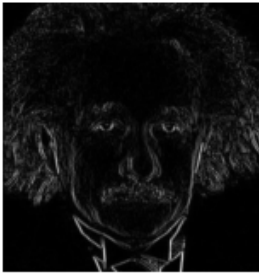
Gy_b (Vertical Gradients)



G_b (Overall Edge Magnitude)



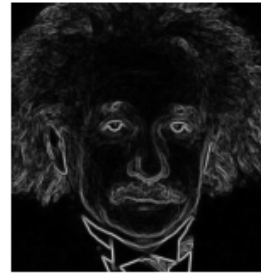
Gx_c (Horizontal Gradients)



Gy_c (Vertical Gradients)



G_c (Overall Edge Magnitude)



0.1.8 Question 8

```
[ ]: def zoom_image(im, s, method='bilinear'):
    H, W = im.shape[:2]
    H_new, W_new = int(round(H * s)), int(round(W * s))

    # Grid of output coordinates
    r_out, c_out = np.meshgrid(np.arange(H_new), np.arange(W_new),
    ↪ indexing='ij')

    # Map back to original coordinate system
    r_in = r_out / s
    c_in = c_out / s

    if method == 'nearest':
        r_nearest = np.clip(np.round(r_in).astype(int), 0, H - 1)
        c_nearest = np.clip(np.round(c_in).astype(int), 0, W - 1)
        return im[r_nearest, c_nearest]

    elif method == 'bilinear':
        # Floor and clip coordinates safely within boundaries
        r1 = np.clip(np.floor(r_in).astype(int), 0, H - 1)
        c1 = np.clip(np.floor(c_in).astype(int), 0, W - 1)
        r2 = np.clip(r1 + 1, 0, H - 1)
        c2 = np.clip(c1 + 1, 0, W - 1)
```

```

    # Calculate fractional offsets
    dr = r_in - np.floor(r_in)
    dc = c_in - np.floor(c_in)

    # Expand dimensions for multi-channel (RGB) support
    if im.ndim == 3:
        dr = dr[..., None]
        dc = dc[..., None]

    # Sample 4 surrounding neighbors
    I11 = im[r1, c1]
    I12 = im[r1, c2]
    I21 = im[r2, c1]
    I22 = im[r2, c2]

    # Bilinear interpolation equation
    interpolated = (
        (1 - dr) * (1 - dc) * I11 +
        dr * (1 - dc) * I21 +
        (1 - dr) * dc * I12 +
        dr * dc * I22
    )
    return np.clip(interpolated, 0, 255).astype(np.uint8)
else:
    raise ValueError("Method must be 'nearest' or 'bilinear'")

def compute_normalized_ssd(im_orig, im_reconstructed):

    # Computes Normalized Sum of Squared Differences (NSSD) between two
    ↪ equal-sized images.

    if im_orig.shape != im_reconstructed.shape:
        raise ValueError(f"Shape mismatch: {im_orig.shape} vs {im_reconstructed.
        ↪ shape}. "
                        "Images must have identical dimensions to calculate
        ↪ SSD.")

    orig_f = im_orig.astype(np.float64)
    rec_f = im_reconstructed.astype(np.float64)

    diff = orig_f - rec_f
    return np.sum(diff**2) / np.sum(orig_f**2)

```

```

[ ]: # Load ground truth original image
im_orig = cv2.imread('image.png', cv2.IMREAD_GRAYSCALE)

```

```

# Downscale to create low-res image (s = 1/4)
im_small = zoom_image(im_orig, s=0.25, method='bilinear')

# Upscale small image back to original size (s = 4)
im_zoomed_nn = zoom_image(im_small, s=4.0, method='nearest')
im_zoomed_bl = zoom_image(im_small, s=4.0, method='bilinear')

# Handle off-by-one pixel rounding differences (if any)
if im_zoomed_bl.shape != im_orig.shape:
    im_zoomed_nn = cv2.resize(im_zoomed_nn, (im_orig.shape[1], im_orig.
↪shape[0]))
    im_zoomed_bl = cv2.resize(im_zoomed_bl, (im_orig.shape[1], im_orig.
↪shape[0]))

# Compute Normalized SSD
ssd_nn = compute_normalized_ssd(im_orig, im_zoomed_nn)
ssd_bl = compute_normalized_ssd(im_orig, im_zoomed_bl)

print(f"Normalized SSD (Nearest-Neighbor): {ssd_nn:.6f}")
print(f"Normalized SSD (Bilinear):          {ssd_bl:.6f}")

```

0.1.9 Question 9

```

[ ]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load flower image
im_flower = cv2.imread('fig8_flower.png')
h, w = im_flower.shape[:2]

# (a) Segment flower using GrabCut
rect = (int(w * 0.1), int(h * 0.1), int(w * 0.8), int(h * 0.8))
mask = np.zeros((h, w), np.uint8)
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)

cv2.grabCut(
    im_flower,
    mask,
    rect,
    bgdModel,
    fgdModel,
    iterCount=5,
    mode=cv2.GC_INIT_WITH_RECT
)

```

```

# Extract foreground mask: keep PR_FGD (3) and FGD (1)
fg_mask = np.where((mask == 1) | (mask == 3), 1, 0).astype('uint8')

# (b) Produce Bokeh background blur composite
im_blurred_bg = cv2.GaussianBlur(im_flower, (51, 51), 0)

fg_mask_3ch = fg_mask[:, :, np.newaxis]
im_enhanced = im_flower * fg_mask_3ch + im_blurred_bg * (1 - fg_mask_3ch)

# Display results
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(cv2.cvtColor(im_flower, cv2.COLOR_BGR2RGB))
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(fg_mask, cmap='gray')
axes[1].set_title('GrabCut Mask')
axes[1].axis('off')

axes[2].imshow(cv2.cvtColor(im_enhanced, cv2.COLOR_BGR2RGB))
axes[2].set_title('Bokeh Composite (Enhanced)')
axes[2].axis('off')

plt.tight_layout()
plt.show()

```

0.1.10 Question 10

```

[ ]: import cv2
import numpy as np

# Load Lake Minaret grayscale image
im_lake = cv2.imread('fig9_lake.png', cv2.IMREAD_GRAYSCALE)

# Standard parameters
d = 9 # Kernel diameter (9x9)
sigma_s = 15.0 # Spatial sigma
sigma_r = 30.0 # Range/intensity sigma

# (a) OpenCV Bilateral Filter vs Gaussian Blur
im_gaussian = cv2.GaussianBlur(im_lake, (d, d), sigmaX=sigma_s)
im_bilateral_cv = cv2.bilateralFilter(im_lake, d=d, sigmaColor=sigma_r,
    ↪sigmaSpace=sigma_s)

# (b) Custom Bilateral Filter Implementation

```

```

def custom_bilateral_filter(image, d, sigma_s, sigma_r):
    radius = d // 2
    # Pad image with reflection to handle boundaries seamlessly
    padded = np.pad(image, radius, mode='reflect').astype(np.float64)
    output = np.zeros_like(image, dtype=np.float64)

    # 1. Precompute Spatial Gaussian weights (constant for all pixels)
    y, x = np.mgrid[-radius : radius + 1, -radius : radius + 1]
    spatial_weights = np.exp(-(x**2 + y**2) / (2.0 * sigma_s**2))

    H, W = image.shape
    for i in range(H):
        for j in range(W):
            # Extract neighborhood region
            region = padded[i : i + d, j : j + d]
            center_val = padded[i + radius, j + radius]

            # 2. Compute Range Gaussian weights dynamically
            range_weights = np.exp(-((region - center_val) ** 2) / (2.0 *
↪sigma_r**2))

            # 3. Combine spatial and range weights
            total_weights = spatial_weights * range_weights
            output[i, j] = np.sum(region * total_weights) / np.
↪sum(total_weights)

    return np.clip(output, 0, 255).astype(np.uint8)

# Run custom bilateral filter with identical parameters
im_bilateral_custom = custom_bilateral_filter(im_lake, d=d, sigma_s=sigma_s,
↪sigma_r=sigma_r)

# (c) Quantitative Evaluation (MSE & PSNR)
mse_val = np.mean((im_bilateral_cv.astype(np.float64) - im_bilateral_custom.
↪astype(np.float64)) ** 2)

if mse_val == 0:
    psnr_val = float('inf')
else:
    psnr_val = 10 * np.log10((255.0 ** 2) / mse_val)

print(f'Mean Squared Error (MSE): {mse_val:.4f}')
print(f'Peak Signal-to-Noise Ratio (PSNR): {psnr_val:.2f} dB')

```